

CareerLens

CareerLens is a student-focused career platform for resume review, job matching, career planning, and interview preparation. The repository currently contains a React frontend and a FastAPI backend with the core data models and API route structure in place.

Problem It Solves

CareerLens is intended to help students understand resume quality, compare their profile with job requirements, identify career skill gaps, follow a learning roadmap, and practice interviews in one application.

Current Features

Implemented in the current codebase:

- Public landing page with login and registration routes.
- Student-facing routes for dashboard, skills, resume upload/analysis, jobs, career coaching, career roadmap, interview preparation, profile, and settings.
- User registration and login with bcrypt password hashing and JWT bearer tokens.
- Authenticated profile retrieval and profile name/email updates.
- Dashboard metrics endpoint with readiness and ATS score fields.
- PDF and DOCX resume upload and per-user resume listing/retrieval.
- Interview session creation and retrieval.
- API route scaffolding for resume analysis, job matching, career analysis/roadmaps, and interview answer evaluation.

The following areas are currently placeholders rather than completed functionality: ATS scoring, extracted resume content, job recommendations, semantic job matching, career skill-gap analysis, roadmap generation, interview question generation, and answer feedback. The frontend service modules also currently return local mock data for most operations instead of calling the backend.

Tech Stack

- Frontend: React 19, React Router, Vite, Axios, Recharts, Lucide React, OGL, Tailwind CSS, PostCSS, and Oxlint.
- Backend: Python, FastAPI, Uvicorn, Pydantic, pydantic-settings, SQLAlchemy, Psycpg, Passlib, and python-jose.
- Database: PostgreSQL accessed through SQLAlchemy and Psycpg.
- Testing: Pytest and FastAPI `TestClient`.

Project Structure

```
CareerLens/  
├── backend/
```

```

├── main.py                # FastAPI application and router registration
├── config.py              # Environment-backed application settings
├── database.py            # SQLAlchemy engine and session setup
├── models.py              # Database models
├── schemas.py             # Pydantic request and response schemas
├── crud.py                # Database operations
├── security.py            # Password hashing and JWT helpers
├── routers/
├── interview APIs
├── └── tests/              # Backend tests
├── frontend/
├── └── src/App.jsx         # React route definitions
├── └── src/pages/         # Application pages
├── └── src/components/    # Shared UI, navigation, and landing components
├── └── src/services/      # API client and feature service modules
└── docs/phase-1/         # Product requirements and workflow documents

```

Backend and Database

The backend creates the SQLAlchemy tables on application startup. The current models include users, resumes, ATS results, jobs, applications, and interview sessions. Authentication-protected routes use bearer tokens issued by the auth endpoints.

The configured database URL uses PostgreSQL. There are no migration files in the repository, so schema changes are currently handled by the startup table-creation call rather than a migration tool.

API Endpoints

The backend runs with the `/api/v1` prefix for feature routes. Protected endpoints require an `Authorization: Bearer <token>` header.

Method	Endpoint	Purpose
GET	/	Backend health response ({"status": "ok"})
POST	/api/v1/auth/register	Register a user
POST	/api/v1/auth/login	Authenticate and issue a JWT
GET	/api/v1/users/me	Get the authenticated user
PUT	/api/v1/users/me	Update the authenticated user's name or email
GET	/api/v1/users/me/dashboard	Get dashboard metrics
POST	/api/v1/resumes/upload	Upload a PDF or DOCX resume
GET	/api/v1/resumes/	List the authenticated user's resumes
GET	/api/v1/resumes/{resume_id}	Get one of the user's resumes

Method	Endpoint	Purpose
GET	<code>/api/v1/resumes/{resume_id}/analysis</code>	Get resume analysis fields; currently returns empty/default results
POST	<code>/api/v1/matching/analyze</code>	Analyze a resume against a job description or job ID; currently returns zero/empty results
GET	<code>/api/v1/matching/jobs?resumeId={id}</code>	Get job matches; currently returns an empty list
POST	<code>/api/v1/career/analyze</code>	Analyze career skill gaps; currently returns an empty roadmap
POST	<code>/api/v1/career/roadmap</code>	Create a career roadmap; currently returns an empty roadmap
POST	<code>/api/v1/interview/start</code>	Create an interview session
POST	<code>/api/v1/interview/evaluate</code>	Evaluate an answer; currently returns a zero score and empty feedback
GET	<code>/api/v1/interview/{session_id}</code>	Get an interview session

Setup and Installation

Prerequisites

- Python with support for the versions required by `backend/requirements.txt`.
- Node.js and npm.
- A running PostgreSQL instance and a `careerlens` database, unless `DATABASE_URL` is changed.

Backend

From the repository root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r backend\requirements.txt
```

Create `backend/.env` from `backend/.env.example` and set a PostgreSQL connection string and a private JWT secret. Start the API from the repository root:

```
python -m uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

In a second terminal:

```
cd frontend
npm install
npm run dev
```

The Vite development server uses its normal local URL, which is typically `http://localhost:5173`.

Environment Variables

Backend (`backend/.env`)

The backend reads these settings from `backend/.env`:

Variable	Purpose	Default in code
<code>DATABASE_URL</code>	PostgreSQL connection URL	Local <code>careerlens</code> database URL
<code>JWT_SECRET_KEY</code>	JWT signing secret	Development value in <code>config.py</code>
<code>JWT_ALGORITHM</code>	JWT signing algorithm	<code>HS256</code>
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Access-token lifetime	<code>30</code>
<code>FRONTEND_URL</code>	Allowed frontend origin for CORS	<code>http://localhost:5173</code>
<code>UPLOAD_DIR</code>	Resume upload directory	<code>uploads</code>

Frontend (`frontend/.env`)

- `VITE_API_BASE_URL` is optional. If set, it replaces the default API URL `http://localhost:8000/api/v1`.

How to Run

1. Start PostgreSQL and configure `backend/.env`.
2. Start the backend with Uvicorn on port `8000`.
3. Start the frontend with `npm run dev` in `frontend/`.
4. Open the frontend development URL in a browser.

Current Status

This is an early working scaffold. The application structure, UI routes, persistence models, authentication flow, resume upload route, and several protected API contracts exist. The feature-specific intelligence and production integrations are not implemented yet, and the frontend is mostly presented as an API-ready interface with mock service responses.

Known Issues

- Frontend authentication, resume, job, career, interview, and user services mostly use simulated responses; their Axios calls are commented out.
- Resume analysis, matching, career, and interview evaluation endpoints return default or empty values.
- Job listings and application management are not implemented as working API workflows.
- The backend has no migration system and creates tables at startup.
- The backend configuration contains development defaults for the database URL and JWT secret; production deployments must override them.
- Running `python -m pytest backend/tests` from the repository root currently fails during test collection because the test modules use package-relative imports but `backend/tests` has no package initializer.
- Role values are stored on users, but the current dependency layer does not enforce role-based authorization.

Future Improvements

Planned improvements based on the phase-one project documentation and current placeholders include:

- Connect frontend services to the live backend APIs.
- Implement real resume text extraction and ATS scoring with actionable recommendations.
- Add job data, job recommendations, semantic matching, and application tracking.
- Implement target-role skill-gap analysis and personalized learning roadmaps.
- Generate categorized interview questions and evaluate answers against rubrics.
- Add active-resume management and complete dashboard progress metrics.
- Add role-based access control, database migrations, stronger production configuration, and broader automated test coverage.